

dr. Matej Šprogar

# Algoritmi in programi

Zakaj jih rabimo in zakaj so tako čudni...



# Pogled na svet

Programiranje je lažje, če ne rabimo poznati ali lahko celo ignoriramo večino **podrobnosti** o delovanju računalnika:

1. Kako uporabljamo systemske vire?
  - a. rezervacija, sproščanje pomnilnika
  - b. delo z datotekami, s strojnimi napravami...
2. Kako so podatki shranjeni?
3. Kako deluje procesor?
4. ...

Na probleme želimo gledati s čim *višjega* gledišča, brez podrobnosti, ki odvrtaajo pozornost od principa rešitve.

*Primer: Da bi recimo sešteli  $a$  in  $b$  ne rabimo poznati strojnih ukazov in registrov, lažje je po **domače** zapisati  $a+b$ , pa naj potem prevajalnik poskrbi za vse detajle...*

Če je le mogoče, naj vsa (nadležna) stranska opravila za nas opravijo drugi programi, predvsem **prevajalnik** in **programske knjižnice**.

# Algoritem

Principu, kako rešimo problem, pravimo tudi *algoritem*. Gre za splošen opis postopka rešitve v obliki končnega nabora operacij, ki ob izvajanju opravijo neko nalogo v omejenem številu korakov.

Algoritem je opis rešitve na tim. **abstraktnem** nivoju. Če ga želimo uporabiti v računalniku, ga moramo pretvoriti v programsko obliko (za konkretno platformo/procesor).

Algoritem lahko izrazimo

- (1) s človeškim jezikom,
- (2) grafično,
- (3) s psevdo-kodom ali
- (4) s programskim jezikom.

Za mnogo splošnih tipov problemov so že znani dobri ali celo *optimalni* algoritmi, ki so pogosto na razpolago v okviru programskih knjižnic.

# 1. Opis algoritma z naravnim jezikom

Takšen zapis je izredno abstrakten in nedorečen, zajema ogromno izkušenj in znanja, česar procesor enostavno nima.

Upravljanje računalnika na ta način je, kljub pojavi generativne umetne inteligence, še vedno v domeni znanstvene fantastike in Hollywooda.

Navodilo *"Če si žejen vzami pivo."* v bistvu pomeni ogromno stvari.

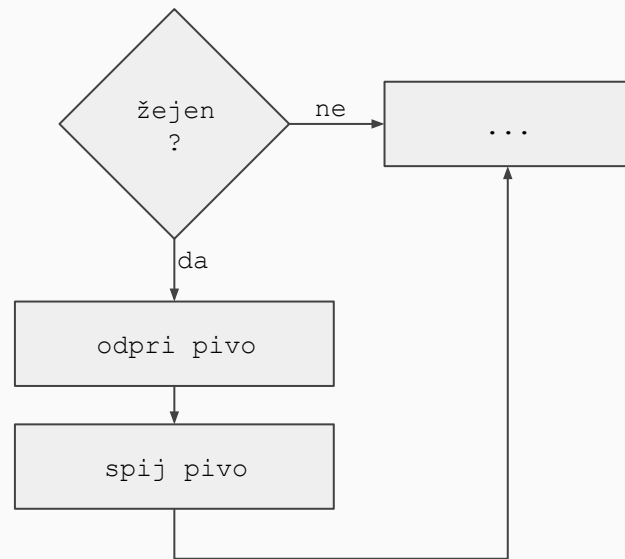
Dejansko to vključuje znanje kaj je pivo, kaj je žeja, čemu pijemo, kako pijemo, kdaj pijemo, koliko pijemo... in seveda še praktična znanja (tekočina teče navzdol, pijemo skozi usta, preveč piva ni dobro...)

## 2. Grafični opis algoritma

Tipično lahko naravni jezik dopolnimo s slikami (slika je(?) 1000 besed), ki vizualno predstavijo neke odvisnosti in odnose med elementi.

Ta princip z izraznostjo ne odtehta svoje zahtevnosti za uporabo, čeprav obstaja veliko poskusov izdelave podpornih orodij in ustreznih prevajalnikov. Velike slike so hitro nepregledne, težko razumljive in nedorečene.

Večina slik so tim. **diagrami poteka** (*Flowchart*).



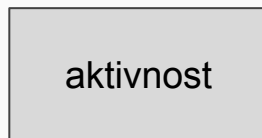
# Diagrami poteka

Osnovni gradniki so:

1. aktivnost (stavek, ukaz, akcija)
2. povezava
3. odločitev (vejitev)
4. terminal (začetek ali konec)

Dodatno še elementi za vnos/izpis, podatke, procese, dokumente, naprave...

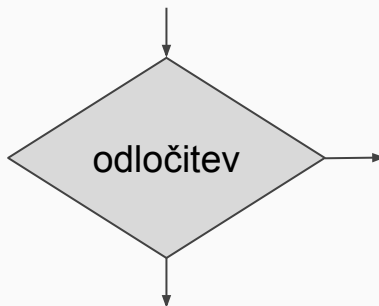
Najpogostejši trije tipi diagramov poteka so (1) Procesni diagram, (2) Diagram pretoka podatkov in (3) Diagram poslovnih procesov.



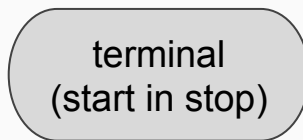
Naloga, ki jo je potrebno opraviti in ki spreminja vsebino, obliko ali lokacijo podatkov



Vrstni red elementov je načeloma od zgoraj navzdol, večinoma je označen tudi s puščico



Odločitev, ki jo je potrebno sprejeti, algoritem se nadaljuje po **eni** izmed **dveh** možnih poti



Vozlišči, ki predstavljata začetek in konec programa ali podprograma



# 3. Opis algoritma s psevdokodom

Pregleden zapis algoritma, ki jasno izraža svoj namen (ne sme vsebovati logičnih napak), ni pa še izvedljiv (manjkajo posebnosti in drobci, ki jih zahteva programski jezik).

Psevdokod je od programskih jezikov neodvisen način zapisa ključnih idej algoritma. Uporaben izključno za komuniciranje med ljudmi.

Algoritem je **berljiv** zaradi svoje *strukturiranosti* na sekvence, odločitve in zanke.

```
če žejen potem  
    vzami pivo  
konec odločitve
```

ali v bolj razgrajeni obliki:

```
če žejen potem  
    odpri pivo  
    spij pivo  
konec odločitve
```

Psevdokod nima *standardizirane* oblike.

## 4. Opis algoritma s programskim jezikom

Ta zapis poleg človeka "razume" še prevajalnik dotičnega jezika.

Ker se programski jeziki razvijajo in so v sorodu, je njihova oblika pogosto zelo zelo podobna, kar pa ne pomeni, da jih lahko enačimo.

Dorečene morajo biti **vse** podrobnosti in odvisnosti (v primeru na desni manjkajo še definicije treh tim. *funkcij*).

**Nič ni prepuščeno naključju.**

```
// primer v jeziku Python
```

```
if zejen():  
    odpri_pivo()  
    spij_pivo()
```

```
// isti primer v C++, Javi in C#
```

```
if (zejen()) {  
    odpri_pivo();  
    spij_pivo();  
}
```

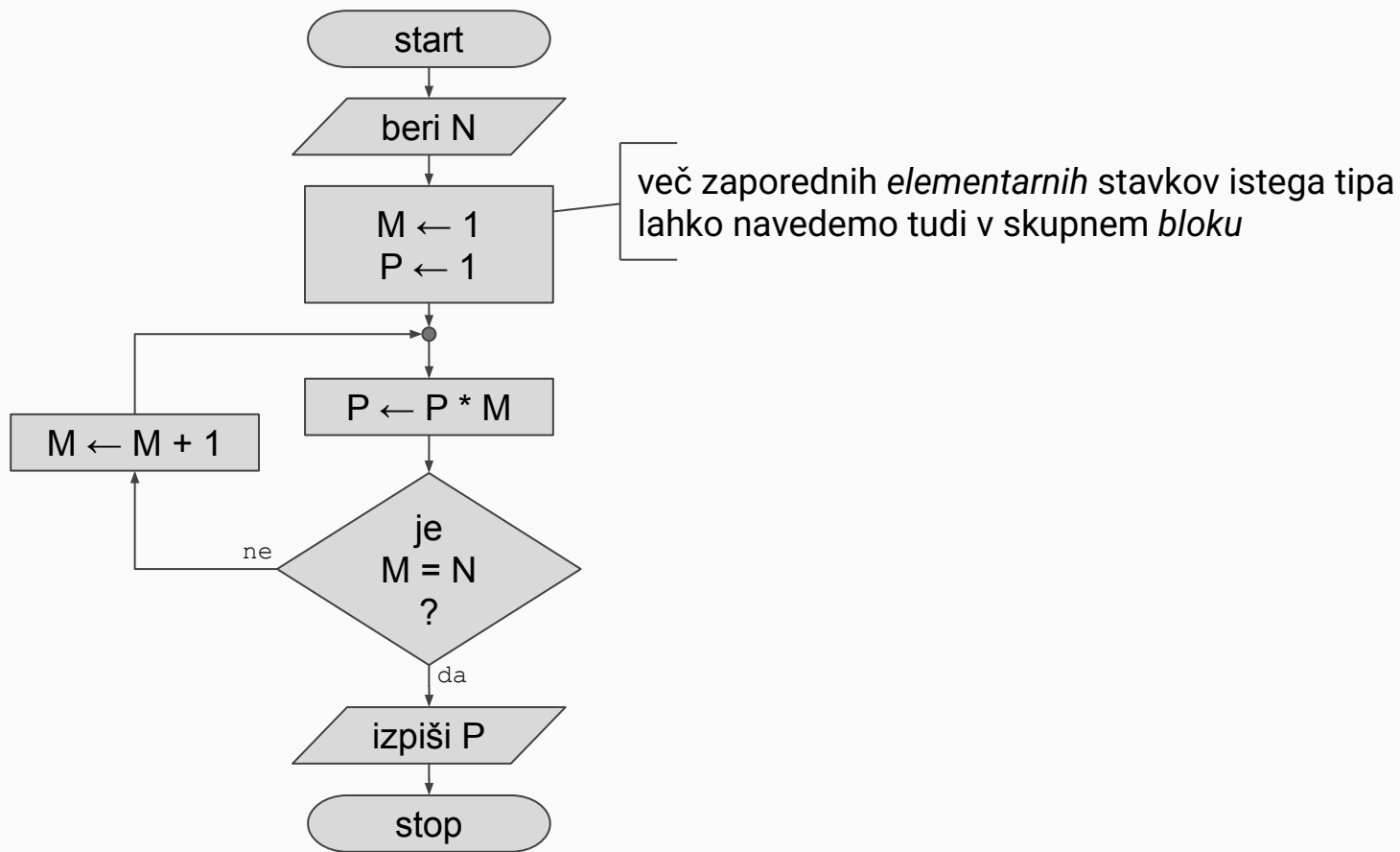


# Paradigme programiranja

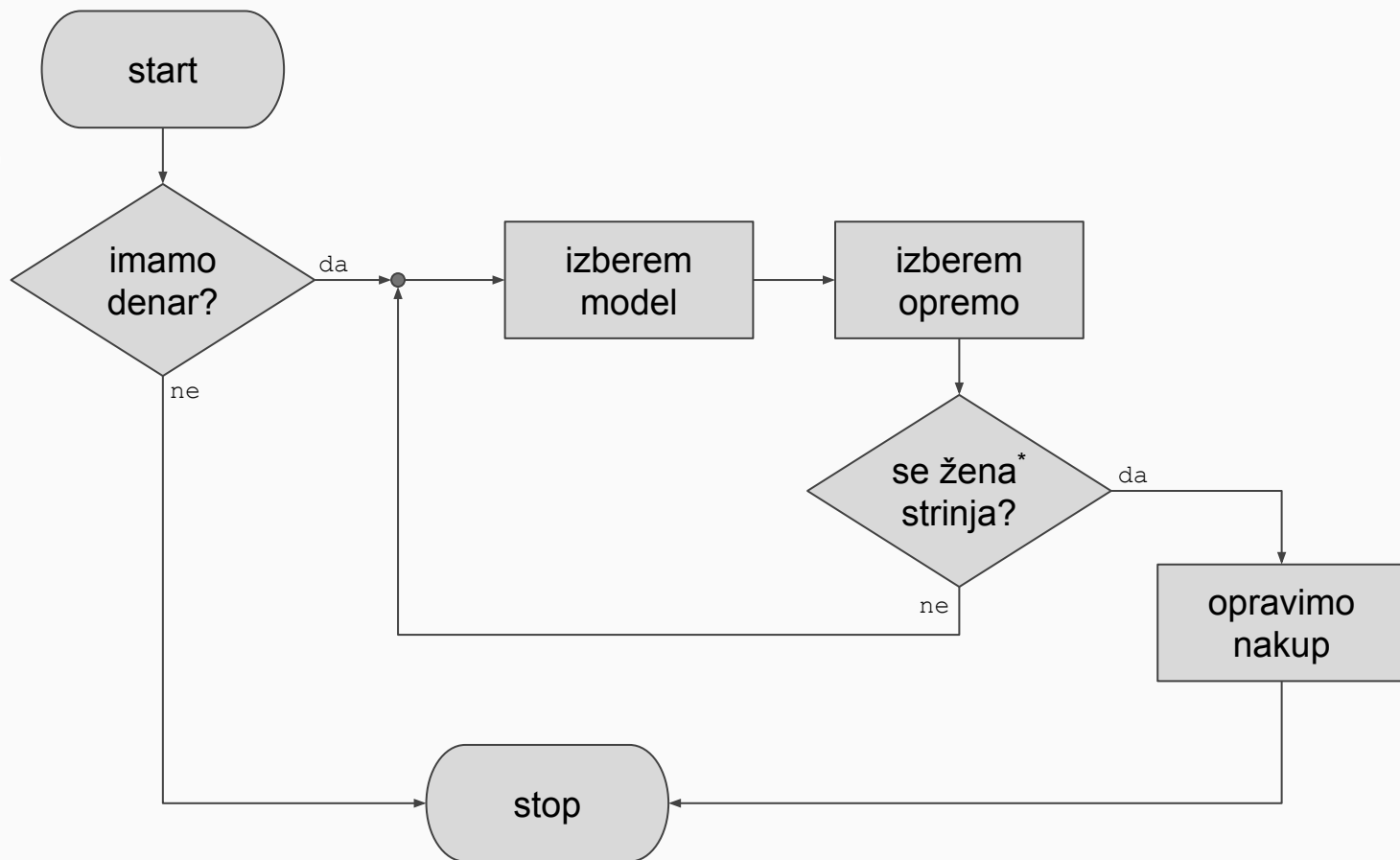
Ker programski jeziki izhajajo iz zmožnosti procesorja, so v osnovi zelo primitivni; algoritme, ki uporabljajo procesorju domače tipe (npr. *int*, *double*, *bool*...), pogosto implementiramo s **strukturiranim programiranjem**.

Ko pa pričnemo v algoritmih razmišljati in delati z elementi človekovega sveta in znanja (npr. *hiša*, *streha*, *vrata*...), sledi **nadgradnja** z **objektnim** načinom razmišljanja in programiranja (poseben predmet).

Potem je tu še *funkcijsko programiranje* in ...



# Primer



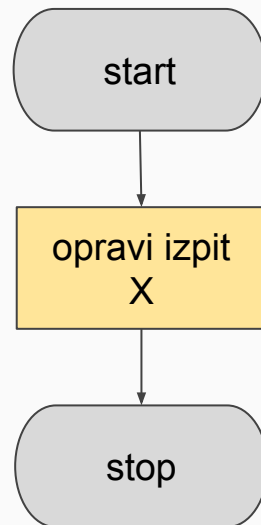
# Naloga

Zapiši algoritem, ki opisuje:

- **opravljanje izpita drugod**
- **opravljanje tega izpita**

PAZI:

1. teorijo lahko opraviš ali s kolokviji ali na rednem izpitnem roku
2. vaje **in** teorija obvezno  $\geq 50\%$
3. največ 6 uradnih poskusov



# Zaključek

Programski jeziki skušajo človeku olajšati programiranje, hkrati pa so vezani na stroj, ki ga krmilijo. Ker procesor ne naredi nič sam od sebe, morajo jeziki premostiti problem človekovega neznanja strojnega jezika in vseh podrobnosti, ki jih zahteva procesor. Programer mora znati poenostaviti algoritem do točke, da ga lahko *enostavno zapiše s programskim jezikom in elementi, ki jih jezik ponuja*.

Noben jezik ni idealen in trenutna raznolikost programskih jezikov je hkrati dokaz, da NI NAJBOLJŠEGA jezika. Na vsakem področju kraljujejo specifični jeziki, ki najbolje naslavljajo potrebe tistega področja.